Ngeletshedzo Mutwanamba u25039769

Amira Ajanaku u25111699

Nontokozo Madela u25677404

Task 1

1.1 Define your event in approximately one page or less. Give it a name, explain what happens there, and identify at least five concrete kinds of event units and at least three kinds of grouping/area that could appear in the Composite tree.

Event name: NeverLand Theme Park

Description:
NeverLand Theme Park is a theme park that hosts rides across themed lands. Guests can move between the lands where each offers a mic of rides, dining and guest services. Each land is entered through a single gate ticket gate. The rides, kiosks and first-aid station all respond differently to the alerts, notices and changes issued by the control center.

Concrete Event Unit Types (leaves):
- RollerCoaster Ride
- Water Ride
- Kids Ride
- FoodKiosk
- TicketGate
- FirstAidStation

Groupings/Area Types (composite):
- Land (Like KidsLand, WaterLand - owns a RideGroup and ServiceGroup)
- RideGroup (Holds all the rides that belong to the land)
- ServiceGroup (Holds the non-ride leaves like foodkiosk, ticketgate and firstaidstation that are in the land – ticketgate is only one per land)

1.2 Identify the GoF participants of both Observer and Composite in your design. If one class participates in more than one role, list each role separately and explain the collaboration in which that role applies.

Composite Participants

- Component -> EventComponent
- Composite -> EventGroup (Land, RideGroup, ServiceGroup)
- Leaf-> Concrete Event Units (RollerCoasterRide, WaterRide, KidsRide, FoodKiosk, TicketGate, FirstAidStation)

Observer Participants

- Subject -> Subject
- Concrete Subject -> EventControl, Land, RideGroup, ServiceGroup
- Observer -> Observer
- Concrete Observers -> Event Groups and Event Units

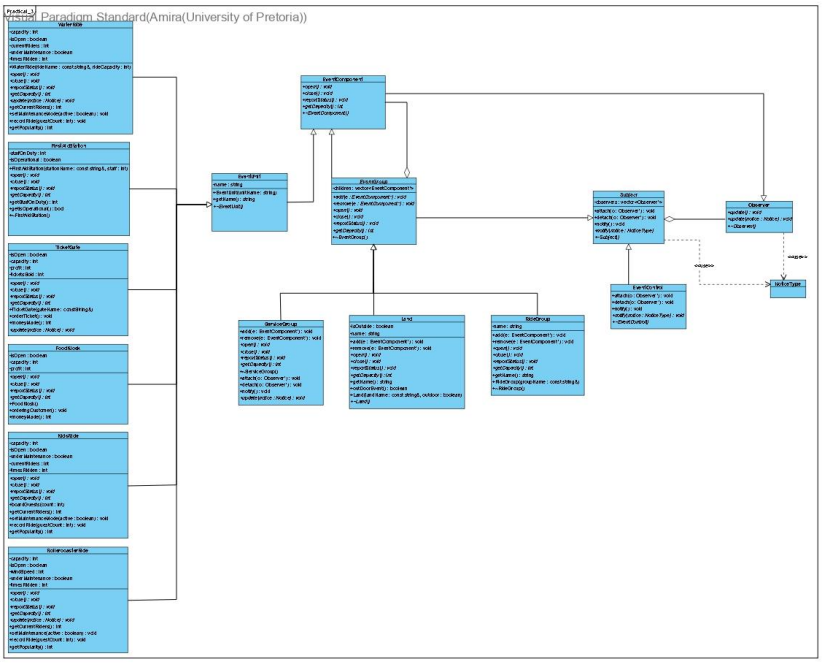
Classes that participate in more than one GoF role are:

- Land
    - **Composite:** structurally contains its RideGroup and ServiceGroup.
    - **Observer:** registers with EventControl so that it can receive park-wide notifications.
    - **Subject:** after receiving an appropriate notice from EventControl, it forwards the notice to interested groups registered below it.
- RideGroup
    - **Composite:** ontains ride objects such as RollerCoasterRide, WaterRide and KidsRide.
    - **Observer:** registers with its containing Land so that it can receive park-wide notifications.
    - **Subject:** can notify its registered rides when a relevant notice is received.
- ServiceGroup
    - **Composite:** contains service leaves such as FoodKiosk, TicketGate and FirstAidStation.
    - **Observer:** registers with its containing Land so that it can receive park-wide notifications.
    - **Subject:** forwards applicable notices to its registered service units.

- Concrete Event Units (RollerCoasterRide, WaterRide, KidsRide, FoodKiosk, TicketGate, FirstAidStation) each is a:
    - **Leaf:** an EventComponent but contains no child components.
    - **Concrete Observer:** reacts when its corresponding Event Group sends it a notice.

1.3 Complete the UML class diagram for your full EventFlow design. Show abstract classes/operations, inheritance, associations, multiplicities, ownership, observer registrations, relevant attributes and operations, and a virtual

destructor on every polymorphic base. Your diagram must include at least five concrete leaf types and enough structure to support every scenario in Task 5.

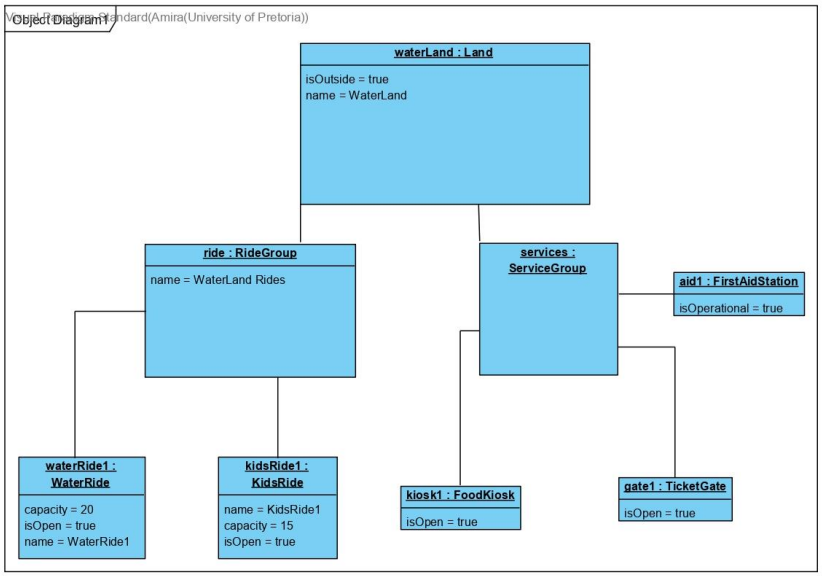


1.4 Write a design rationale. Explain: (a) why your Composite is a genuine part-whole tree; (b) why Observer is required rather than direct hard-coded calls; (c) who owns event components; (d) who owns or does not own observer references; and (e) why any class that is both Subject and Observer is not a misuse of either pattern.

- Composite is a genuine part-whole tree because Land contains groups such as RideGroup and ServiceGroup, which then contain individual event units, allowing operations to work recursively through the park hierarchy.
- Observer is required rather than direct hard-coded calls because EventControl can notify objects through the common Observer interface without knowing every concrete rode or service, reducing coupling and allowing different units to respond polymorphically.
- EventComponents are owned by the Composite that contains them, so deleting the root recursively destroys the complete owned tree exactly once.
- Observer references are non-owning, meaning that a Subject stores pointers to its registered observers but does not delete them, keeping relationships separate from object ownership.
- Because in such a class as ServiceGroup being the two roles apply different relationships, for example as an Observer it the class to receive notifications from the level above and as a Subject forwards them to the registered objects below as a Subject , enabling cascading notifications.

Task 2

2.3 Build a sample event with at least three levels of nesting. Draw an object diagram showing the concrete object instances and the ownership tree.



Task 3

3.5 Choose push or pull for your Observer implementation. Explain the trade-off and show the exact state/message information transferred in your design.

We chose push, because in push the Subject sends the notice directly to each Observer through update(NoticeType notice), so the observers do not need to query the Subject for additional information.

The trade-offs are that push is simpler for our design and makes cascading notifications easier, but the Subject must know what information the observers need and include it in the notification.

The state/message information transferred in our design is a NoticeType value passed direxlty to update(NoticeType notice). he possible messages include WeatherAlert, CapacityAlert, OpenNotice, CloseNotice, PauseNotice, ResumeNotice, EvacuationNotice, and RushHourNotice.

Task 4

4.4 Add at least three original features that are not explicitly required above. They must fit your event and should create interesting interactions worth modelling.

Briefly explain why each feature could be added without turning one class into a god object.

1. Maintenance mode
A ride can be taken offline on its own initiative. While underMaintenance is true, open() refuses — even if triggered by an OpenNotice from the park.

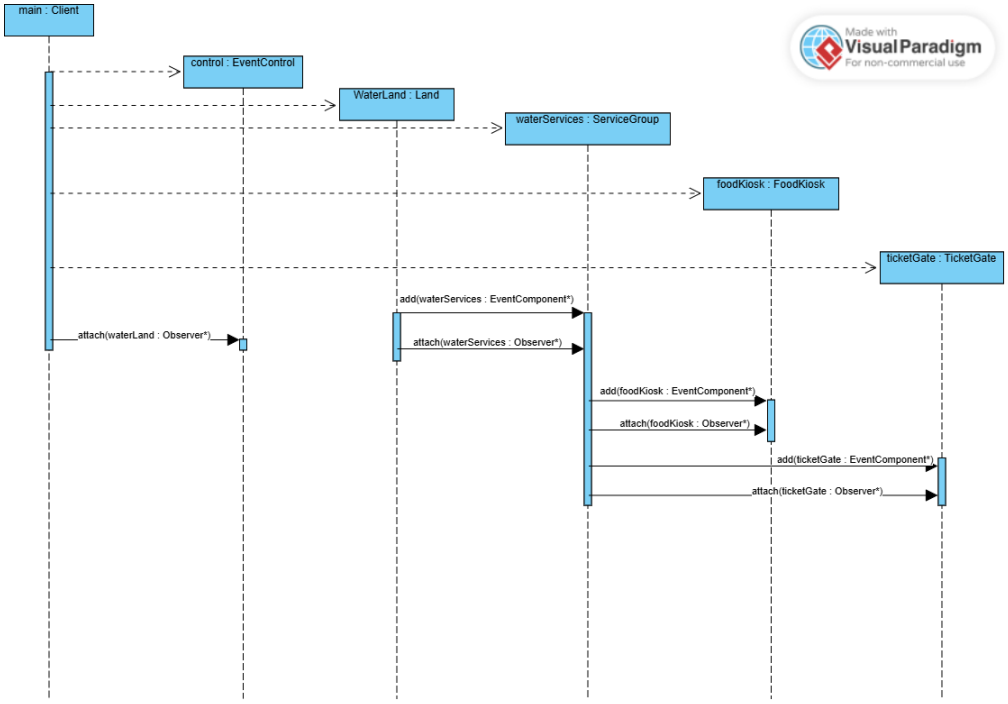
2. Live guest boarding and capacity tracking
Each ride tracks how many guests are currently on it, separate from its fixed capacity. This supports the conditional checks in update()'s CapacityAlert handling, so a ride only closes if it's actually full rather than closing whenever the notice arrives.

3. Ride popularity tracking
Each ride builds up a running total of guests served over its lifetime (timesRidden). This has no dependency on any other class or system and is used to identify the theme park's most popular ride.

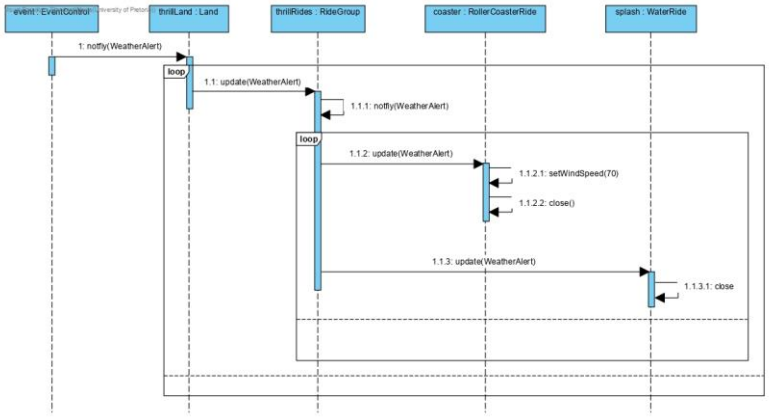
None of these features create a god object because each one is self-contained. They don't reach into another classes' private data, and are all still have the logic of being a ride. The RideGroup EventGroup isn't aware that these features exist as they only interact with rides through the shared EventComponent/Observer interfaces.

Task 5

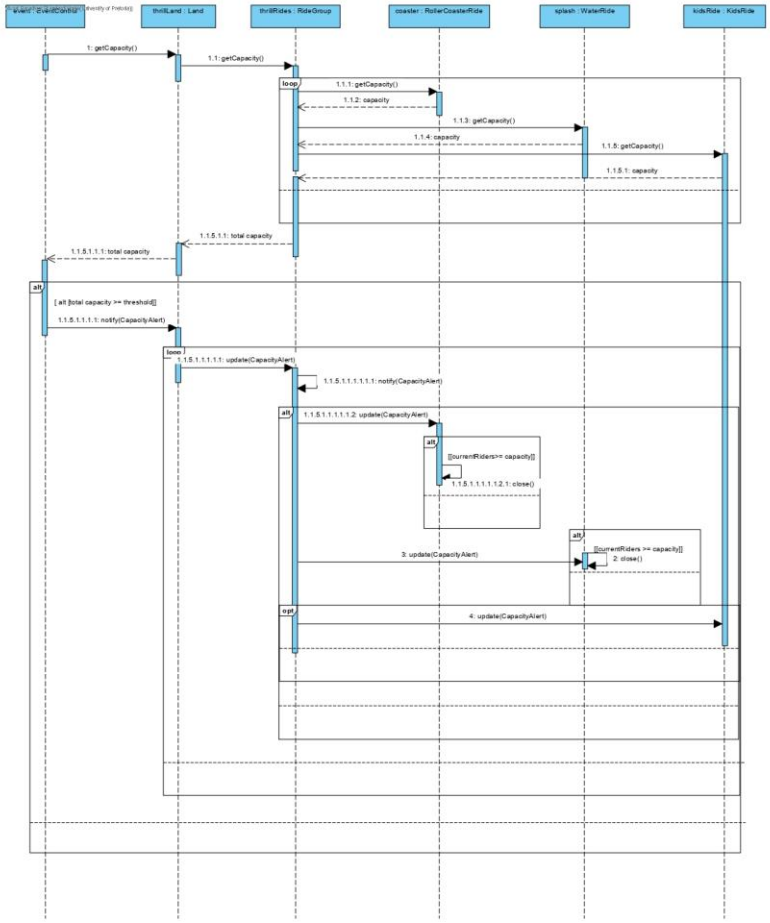
5.1 SD1– Building and registering part of the event. Show the client creating a root Composite, at least one nested Composite and at least two Leaves, adding them to the ownership tree, and establishing the Observer registrations needed for later notifications. Creation messages must begin lifelines at the correct time. The interaction must make it possible for a marker to distinguish Composite ownership from Observer registration.



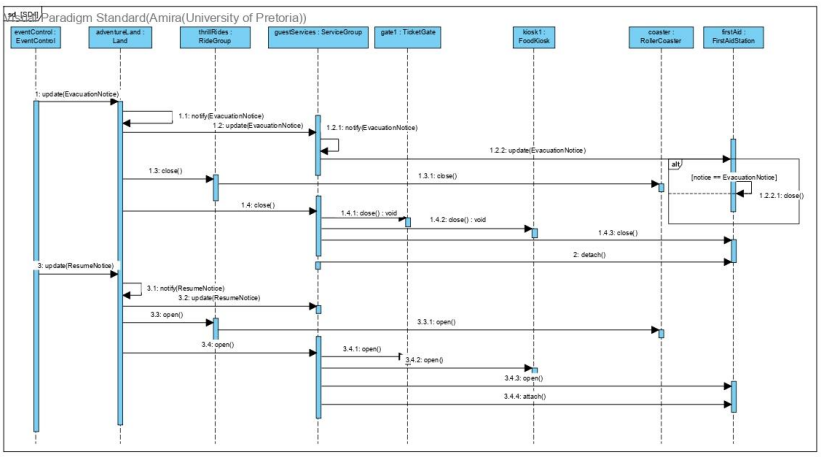
5.2 SD2– Cascading event notification. Show a realistic notice originating from an appropriate Subject and cascading through at least three runtime levels to multiple observers. Use a loop fragment where observers are notified. If you chose pull, show the observer querying the subject; if you chose push, show the information passed to update(...). At least two concrete event units must react meaningfully to the notice.



5.3 SD3– Conditional event response and Composite behaviour. Model a realistic situation such as a capacity threshold, weather warning, schedule disruption, temporary closure or service interruption. The interaction must include a meaningful Composite operation and use an alt fragment with guards. Include an opt fragment where it naturally fits. Show different concrete objects responding through polymorphism rather than explicit type checking.



5.4SD4– Your signature event scenario. Invent a substantial interaction that makes your team's event recognisably different from another team's system. It must involve at least six lifelines, at least two Composite levels, at least one Observer notification, and at least one change during execution, such as an observer joining/leaving, a unit being reassigned, a pause followed by resume, a safety response followed by recovery, or end-of-event shut down. If ownership or registration changes, show the relevant detach/attach or transfer messages clearly. Use at least one combined fragment and ensure the scenario can be traced to your actual C++ implementation.

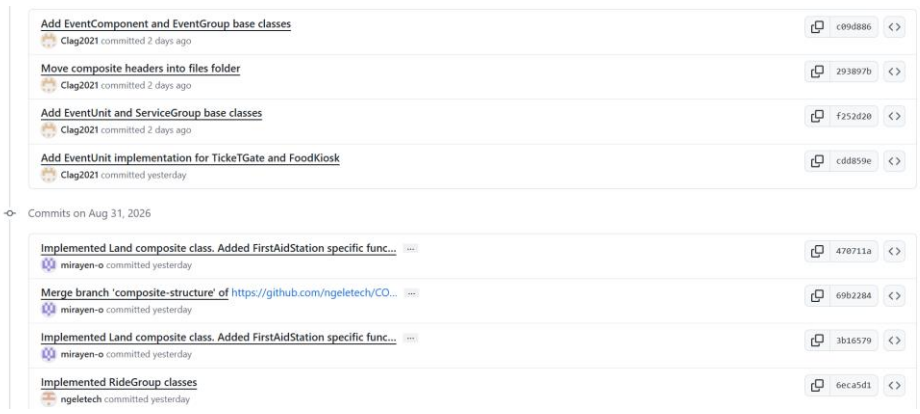


Task 7

7.1 **In your submitted PDF, include a short GitHub workflow reflection (approximately half a page). Explain how the three members divided the work, how changes were integrated, and identify two or three commits or other repository activities that show meaningful collaboration. If your team used branches or pull requests, briefly explain how they helped.**

Our team divided work primarily by individual interest rather than rigid task division. Each member chose which classes and subclasses they would implement and if someone needed help, we asked each other directly rather than working in isolation. If a member noticed that something was incorrect/ missing in one task they stepped in to fix it. Ngele set up the initial project skeleton and implemented the ride related classes. Nontokozo built the base classes and the ServiceGroup/EventUnit side of the Composite structure. Amira implemented Land subclass and FirstAidStation leaf.

We used feature branches to keep, our individual work isolated before merging – composite-structure for the base Composite classes and observer-structure for the Subject/Observer interfaces and classes. Using branches in this way let each of us build and test our work without interfering with each other's work. The work was able to progress in parallel without blocking one another.



Practical_3

Visual Paradigm Standard(Amira(University of Pretoria))